

Database Core Concepts

Why not create one huge table?

That would create:

- duplication
- inconsistency
- wasted storage
- update problems

Internal Working

When MySQL executes

```
CREATE TABLE Student (...);
```

it does much more than create a visual table.

Internally it:

- Checks that the table name is valid.
- Verifies permissions.
- Validates every datatype.
- Validates every constraint.
- Creates metadata in the MySQL data dictionary.
- Creates storage structures for the table (through the storage engine, typically InnoDB).
- Makes the table available for queries.

CREATE TABLE Syntax

Basic syntax:

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype  
);
```

Example:

```
CREATE TABLE Student (  
    StudentID INT,  
    Name VARCHAR(100),  
    CGPA DECIMAL(3,2)  
);
```

Columns

A column represents one attribute of every record.

Rows

A row represents one complete record.

Example:

StudentID	Name	CGPA
1	Aadya	7.97

Properties of a Primary Key

A primary key:

- must be unique,
- cannot be NULL,
- there can be only one primary key definition per table (though it may contain multiple columns).

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(100)  
);
```

Internal Working

When a primary key is created, InnoDB automatically creates a clustered index on it. This allows rows to be located efficiently. We'll study indexes in detail later.

NOT NULL

Normally, columns can contain NULL unless specified otherwise.

Example:

```
CREATE TABLE Student (  
    Name VARCHAR(100) NOT NULL  
);
```

Now every inserted row must provide a name.

Attempting:

```
INSERT INTO Student (Name)VALUES (NULL);
```

DEFAULT

Suppose most students are from India.

Instead of writing "India" every time:

```
CREATE TABLE Student (  
    Country VARCHAR(50) DEFAULT 'India'  
);
```

AUTO_INCREMENT

One of the most frequently used MySQL features.
Instead of manually assigning IDs:

1
2
3
4
5

MySQL can generate them.

Example:

```
CREATE TABLE Student (  
    StudentID INT AUTO_INCREMENT PRIMARY KEY,  
    Name VARCHAR(100)  
);
```

Now:

```
INSERT INTO Student (Name)VALUES ('Aadya');
```

creates:

StudentID	Name
1	Aadya

Internal Working

MySQL maintains an internal counter for the AUTO_INCREMENT column.
Each successful insert gets the next available value.

SQL Server Equivalent

SQL Server uses:

```
IDENTITY(1, 1)
```

instead of AUTO_INCREMENT.

Composite Primary Key

Sometimes one column isn't enough.

Example:

A student can enroll in many courses.

Neither StudentID nor CourseID alone is unique.

But together they are.

Neither StudentID nor CourseID alone is unique.

But together they are.